IoT INTEGRATED BUCK CONVERTER

A MINI PROJECT REPORT FOR THE COURSE

231EEC513L – Embedded System Laboratory

Submitted by

HARISH M **(310623105016)**

In partial fulfillment for the award of the degree of

BACHLEOR OF ENGINEERING
IN
ELECTRICAL AND ELECTRONICS ENGINEERING

EASWARI ENGINEERING COLLEGE(AUTONOMOUS), RAMAPURAM

ANNA UNIVERSITY, CHENNAI 600 025

TABLE OF CONTENTS:

Abstract

This project presents the design, development, and implementation of a smart buck converter system integrated with an Arduino / ESP32 microcontroller and a Processing-based graphical dashboard for real-time monitoring and control of key electrical parameters. A buck converter—commonly referred to as a step-down DC-DC converter — efficiently reduces a higher input DC voltage to a stable, lower output voltage suitable for powering electronic devices and circuits. Beyond conventional voltage regulation, this project enhances the converter's functionality by incorporating real-time data acquisition and visualization using embedded systems.

The microcontroller continuously measures input voltage (V_{in}), output voltage (V_{out}), output current (I_{out}), and the potentiometer's position, which controls the output voltage level. To ensure safe and accurate readings, input and output voltages are scaled through resistor voltage dividers before being processed by the analog-to-digital converters (ADCs). The output current is monitored via an ACS712 Hall-effect current sensor, enabling precise and noise-immune current measurement.

Measured parameters are transmitted through *serial communication* to a connected PC, where a custom *Processing-based graphical dashboard* displays the data dynamically. The dashboard features an intuitive interface with labeled, real-time updating boxes for each parameter, offering clear insights into system behavior. Simultaneously, a *16x2 I2C LCD module* provides local on-device feedback, allowing parameter visualization without PC dependency. This dual-display system enhances the versatility and usability of the setup, enabling both standalone and computer-assisted monitoring modes.

Overall, this project serves as a practical and interactive platform for students and engineers to explore the integration of hardware and software in embedded power applications. It effectively connects theoretical understanding with hands-on experimentation, highlighting the growing importance of intelligent monitoring systems in modern electrical engineering.

1. Introduction

In modern electrical and electronic systems, maintaining precise voltage regulation and ensuring real-time monitoring have become fundamental for achieving operational efficiency, safety, and reliability. With the increasing integration of power electronics in industrial, consumer, and research applications, systems are expected not only to regulate electrical parameters effectively but also to provide immediate feedback and data for analysis. Among the wide range of power conversion techniques, the *buck converter*—a DC-DC step-down converter—remains one of the most commonly used due to its simplicity, high efficiency, and adaptability in low-voltage applications.

Traditional buck converters primarily focus on delivering a regulated output voltage from a higher DC input. However, in modern contexts, static voltage regulation alone is insufficient.

Engineers and researchers now require *dynamic insights* into circuit performance parameters such as voltage, current, and control inputs to analyze system efficiency, thermal behavior, and load response. Real-time monitoring thus transforms a conventional converter into an *intelligent power system*, capable of providing actionable information for diagnostics, optimization, and automation.

To address this need, the present project introduces a *smart buck converter platform* enhanced with an *embedded microcontroller system* and a *real-time graphical monitoring dashboard*. The hardware foundation consists of a buck converter circuit interfaced with an Arduino or ESP32 microcontroller. The microcontroller captures analog signals representing the input voltage (V_{in}), output voltage (V_{out}), and output current (I_{out}) through calibrated voltage dividers and a Hall-effect current sensor (ACS712). Additionally, a potentiometer allows the user to manually vary the output voltage, giving the system interactive control capability.

The embedded firmware performs continuous acquisition and processing of sensor data, while simultaneously displaying essential parameters on a *16x2 I2C LCD module* for immediate feedback. Beyond local monitoring, the system transmits real-time data to a *Processing-based dashboard* on a computer, enabling graphical visualization of parameters through dynamically updating numerical panels. This real-time data interface promotes a more intuitive understanding of system behavior under varying load and control conditions.

The system's *modular architecture* ensures flexibility for further extensions, such as incorporating wireless connectivity for IoT-based data logging, automated feedback control, or smart energy optimization. Hence, this project not only serves as a demonstration of real-time monitoring in DC-DC conversion but also as a foundation for future intelligent energy systems capable of adaptive and autonomous operation.

2. Objectives

The objectives of this project are structured to address the hardware design, software development, and educational outcomes associated with implementing an embedded buck converter system. Each objective contributes to the overall goal of creating an efficient, real-time monitoring platform that integrates power electronics and embedded control.

2.1 Hardware Objectives

1. **Buck Converter Design:** Develop a high-efficiency buck converter capable of stepping down a 12V DC input to a controlled, stable lower output voltage suitable for various electronic applications.
2. **Sensor Integration:** Interface the ACS712 Hall-effect current sensor for accurate current measurement and implement voltage divider networks for safe and precise measurement of input (V_{in}) and output (V_{out}) voltages within the microcontroller's ADC range.

3. **Potentiometer Control:** Utilize a potentiometer to provide manual control over the output voltage, enabling users to dynamically adjust the converter's output and observe corresponding changes in system parameters.

4. **LCD Display Implementation:** Integrate a 16x2 I2C LCD module to locally display real-time readings of V_{in}, V_{out}, I_{out}, and potentiometer status, allowing on-device monitoring without requiring external software.

2.2 Software Objectives

1. Analog Data Acquisition: Program the Arduino or ESP32 microcontroller to continuously acquire analog signals from the voltage dividers, current sensor, and potentiometer through ADC channels for precise digital conversion.

2. Serial Communication: Establish reliable serial communication between the microcontroller and a PC to transmit real-time data for visualization and logging.

3. Graphical Dashboard Development: Design and implement a Processing-based graphical user interface (GUI) to display voltage, current, and control parameters in real-time using dynamic visual elements.

4. Potentiometer Status Detection Algorithm: Develop and implement a logical algorithm to detect and classify potentiometer adjustments as *increase*, *decrease*, or *stable*, ensuring responsive system feedback on both LCD and dashboard displays.

2.3 Educational and Practical Objectives

1. Applied Learning in Power Electronics: Enhance understanding of real-world applications of embedded systems within power conversion and energy management contexts.

2. Hands-On System Integration: Provide practical exposure to sensor calibration, voltage scaling, ADC interfacing, and serial data communication, fostering applied engineering skills.

3. Interactive System Visualization: Demonstrate the principles of real-time monitoring, interactive control, and graphical visualization—key aspects of IoT-based and smart electronic systems.

3. Components Used

Component	Specification	Function	Quantity
Buck Converter Module	Adjustable DC-DC step-down (0-12V)	Steps down input voltage	1
Arduino Uno / ESP32	Microcontroller with ADC	Processes sensor data and controls outputs	1
ACS712 Current Sensor	5A Hall effect	Measures output current	1
Potentiometer	47 kΩ	Provides user control input	1
LCD Display	16x2 I2C	Displays system parameters locally	1
Resistors	10kΩ, 5.1kΩ	Voltage dividers for ADC safety	2
Jumper Wires	Standard	Connect components	As required
Power Supply	12V DC	Powers the system	1
PC	Processing IDE	Visualizes data via dashboard	1

4. System Overview

4.1 Power Electronics Module

The buck converter reduces the high DC input voltage to a desired stable output. The potentiometer allows dynamic adjustment of the output voltage, making the system interactive and controllable.

4.2 Sensor Module

- **Voltage Measurement**: Voltage dividers scale down Vin and Vout to levels safe for ADC input.
- **Current Measurement**: ACS712 sensor outputs voltage proportional to current.
- **Potentiometer**: Provides analog voltage input for output control.

4.3 Microcontroller Module

The Arduino/ESP32 reads analog signals, determines potentiometer status, displays parameters on the LCD, and sends data to the PC dashboard.

4.4 Processing Dashboard Module

The dashboard provides real-time visual feedback using labeled boxes for Vin, Vout, Iout, and potentiometer status. It parses serial data and updates the display dynamically, offering intuitive interaction and monitoring.

5. Pin Configuration and Connections

The hardware integration of the system involves interconnecting the Arduino Uno, ESP32, and buck converter with supporting sensors and peripherals. The following subsections detail the signal mapping and interfacing used for each module.

5.1 Arduino Pin Mapping

The Arduino Uno is primarily responsible for analog signal acquisition and control input handling. It reads voltage and current feedback through dedicated ADC channels, and the potentiometer provides a variable control voltage to adjust the converter output. Additionally, an I²C-based LCD display is connected for local monitoring, while serial communication over USB allows real-time data observation or debugging through the PC dashboard.

Signal	Arduino Pin	Function	Description
V_{in}	A0	ADC	Voltage divider from buck input
V_{out}	A1	ADC	Voltage divider from buck output
I_{out}	A2	ADC	ACS712 output pin
Potentiometer	A3	ADC	Wiper to ADC; ends to 5V/GND
LCD SDA	SDA	I2C	Data line to LCD
LCD SCL	SCL	I2C	Clock line to LCD
Serial Tx/Rx	USB	UART	To PC dashboard

5.2 ESP32 Pin Mapping

The ESP32 acts as the IoT interface and main processing controller. It receives analog voltage and current data from the buck converter and transmits it wirelessly to the IoT dashboard. The onboard ADC pins (GPIO32–GPIO35) are used for voltage and current sensing. I²C communication is also implemented for optional display output, and its built-in Wi-Fi module enables remote data publishing to the dashboard.

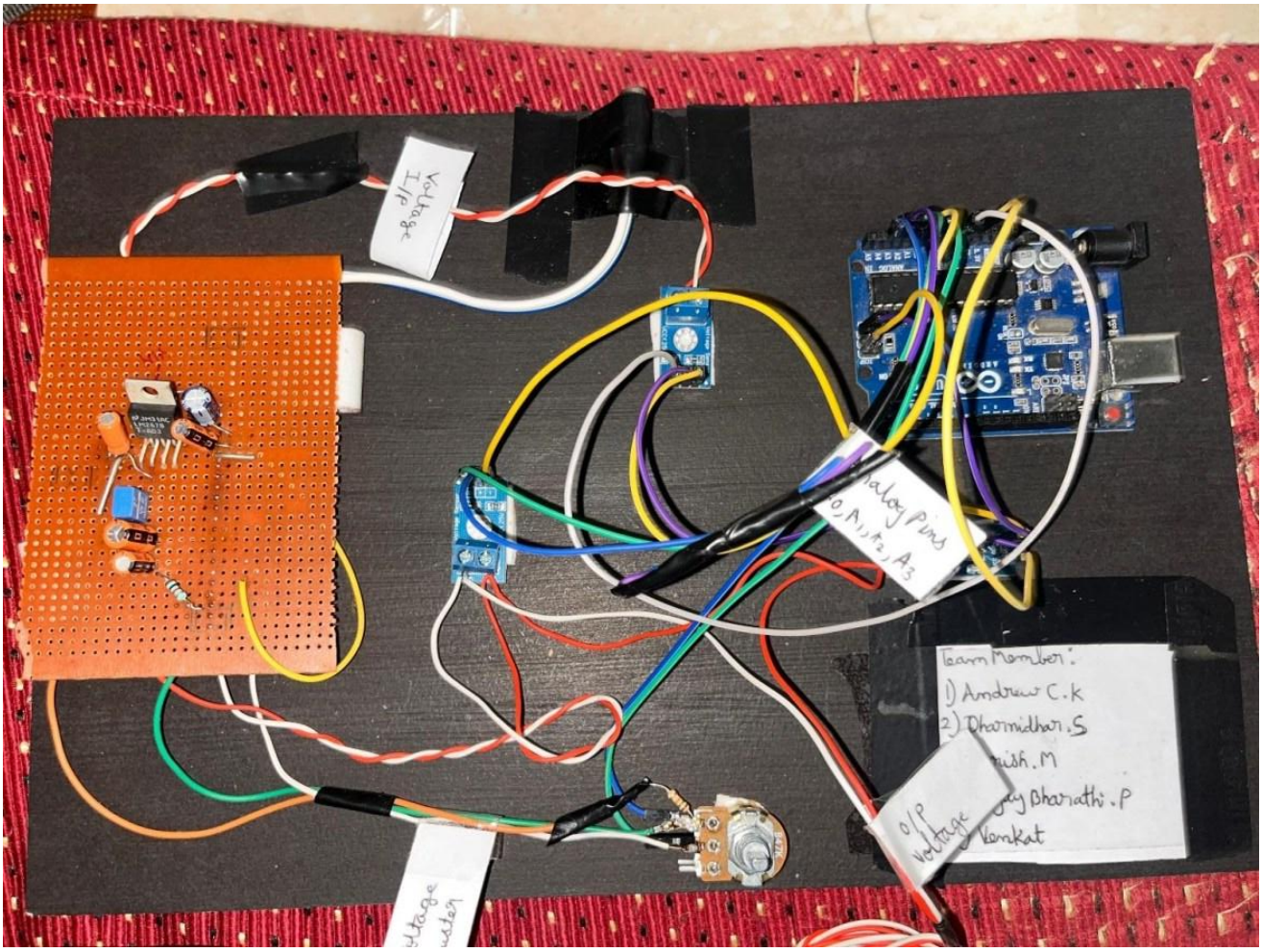
Signal	ESP32 Pin	Function	Description
V_{in}	GPIO34	ADC	From voltage divider
V_{out}	GPIO35	ADC	From voltage divider
I_{out}	GPIO32	ADC	ACS712 output
Potentiometer	GPIO33	ADC	Pot wiper input
LCD SDA	GPIO21	I2C	Data line
LCD SCL	GPIO22	I2C	Clock line

5.3 Buck Converter Connections

The buck converter serves as the core power regulation stage. Its input terminals receive a 12V DC supply, while the output terminals are connected to the load and sensing circuitry. A potentiometer provides manual control for adjusting the duty cycle, which in turn varies the output voltage. The ACS712 current sensor is integrated on the output side to measure the load current and feed the signal to both controllers for monitoring.

Pin	Connection	Description
V_{in+}	12V DC	Input voltage
V_{in-}	GND	Ground reference
V_{out+}	Load +	To voltage divider & ACS712
V_{out-}	Load -	GND, ACS712 output
Adj/Control	Potentiometer	Controls output voltage

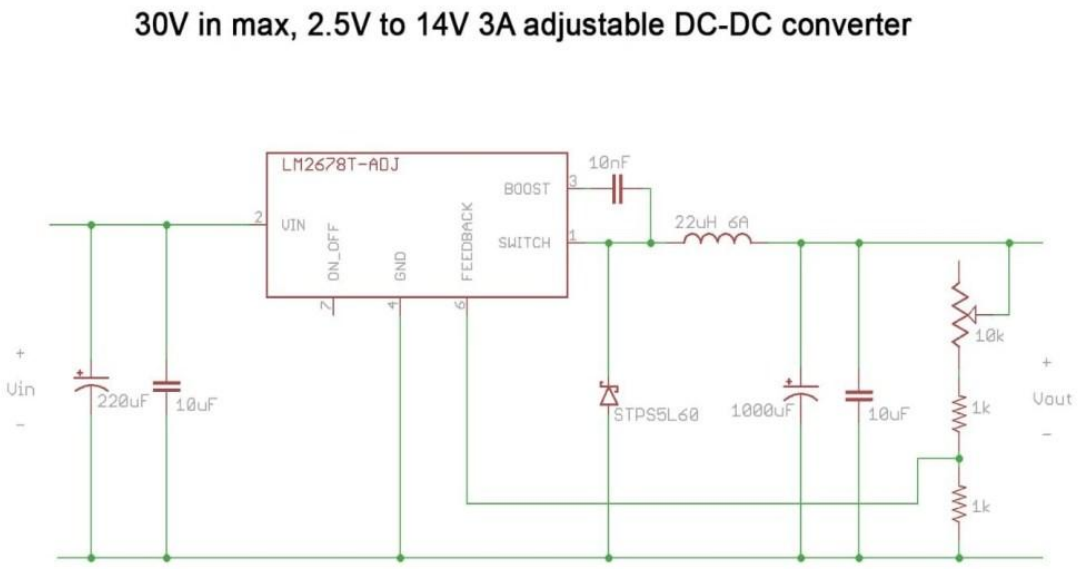
6. Project



In the onverter circuit shown in the schematic. The setup includes the Arduino Uno, ESP32, buck converter module, and an analog control potentiometer. The potentiometer is connected to one of the Arduino's analog input pins to provide a variable reference voltage corresponding to the desired output. The Arduino processes this input and sends the control data to the ESP32 via a serial communication link (UART).

The ESP32, equipped with Wi-Fi connectivity, then reads the corresponding output voltage from the buck converter through an analog sensing line and publishes this data to the IoT dashboard in real time. Power regulators ensure stable 5V and 3.3V supplies for the microcontrollers. All components are mounted on a single board to maintain compactness and facilitate debugging and signal tracing.

7. Circuit Diagram



The circuit diagram focuses on the buck converter hardware implementation. The interfacing between the converter, Arduino Uno, and ESP32 is implemented externally to simplify the schematic.

The Arduino Uno reads the analog control voltage from the potentiometer, processes it, and communicates the corresponding control data to the ESP32 through a serial (UART) connection. The ESP32, acting as the IoT node, transmits the measured output voltage to the online dashboard via Wi-Fi.

Although not shown in the circuit diagram, this modular design reflects the embedded system approach—where power electronics and IoT subsystems can be developed, tested, and iterated independently.

8. Software Implementation

8.1 Arduino Code

```
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ----------------------------
// LCD Setup
// ----------------------------
LiquidCrystal_I2C lcd(0x3F, 16, 2);  // Address 0x27, 16x2 LCD

// ----------------------------
// Pin Configuration
// ----------------------------
#define VIN_PIN    A0  // Input voltage
#define VOUT_PIN   A1   // Output voltage
#define IOUT_PIN   A2   // Output current (ACS712)
#define POT_PIN    A3   // Potentiometer (47kΩ)

// ----------------------------
// Calibration & Variables
// ----------------------------
float prevPotValue = 0;
String potStatus = "Stable";

const float VREF = 5.0;             // Arduino reference voltage
const float VOLT_DIV_RATIO = 5.1; // Adjust for your voltage divider
const float ACS_OFFSET = 2.49;       // ACS712 output offset (V)  const
float ACS_SENSITIVITY = 0.185; // 185 mV/A for 5A sensor

// ----------------------------
// Function to read voltage
// ----------------------------
float readVoltage(int pin) {
  int adc = analogRead(pin);
  float voltage = (adc * VREF) / 1023.0;
  return voltage * VOLT_DIV_RATIO;
}

// ----------------------------
// Function to read current
// ----------------------------
float readCurrent(int pin) {
  int adc = analogRead(pin);
  float voltage = (adc * VREF) / 1023.0;
```

```
  float current = (voltage - ACS_OFFSET) / ACS_SENSITIVITY;
  return current;
}

// ----------------------------//
Detect potentiometer rotation //
-----------------------------
String detectPotRotation(float currentValue) {
  String result;
  if (currentValue > prevPotValue + 5) {
    result = "Increased";
  } else if (currentValue < prevPotValue - 5) {
    result = "Decreased";
  } else {
    result = "Stable";
  }
  prevPotValue = currentValue;
  return result;
}

// ----------------------------
// Setup
// ----------------------------
void setup() {
  Serial.begin(9600);

  lcd.begin();       // Use library-specific begin() with no arguments
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Voltage Monitor");
  delay(1500);
}

// ----------------------------
// Main Loop
// ----------------------------
void loop() {
  float vin = readVoltage(VIN_PIN);
  float vout = readVoltage(VOUT_PIN);
  float iout = readCurrent(IOUT_PIN);
  float potValue = analogRead(POT_PIN);

  potStatus = detectPotRotation(potValue);

  // ---- LCD Display ----
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("IN:");
```

```
  lcd.print(vin, 1);
  lcd.print("V OUT:");
  lcd.print(vout, 1);
  lcd.print("V");

  lcd.setCursor(0, 1);
  lcd.print("I:");
  lcd.print(iout, 2);
  lcd.print("A ");
  lcd.print(potStatus);

  // ---- Serial Monitor ----
  Serial.print("Vin: "); Serial.print(vin, 2); Serial.print("V  ");
  Serial.print("Vout: "); Serial.print(vout, 2); Serial.print("V  ");
  Serial.print("Iout: "); Serial.print(iout, 2); Serial.print("A  ");
  Serial.print("Pot: "); Serial.print(potValue);
  Serial.print(" -> "); Serial.println(potStatus);

  delay(1000);  // Update every second
}
```

8.2 Processing Dashboard Code

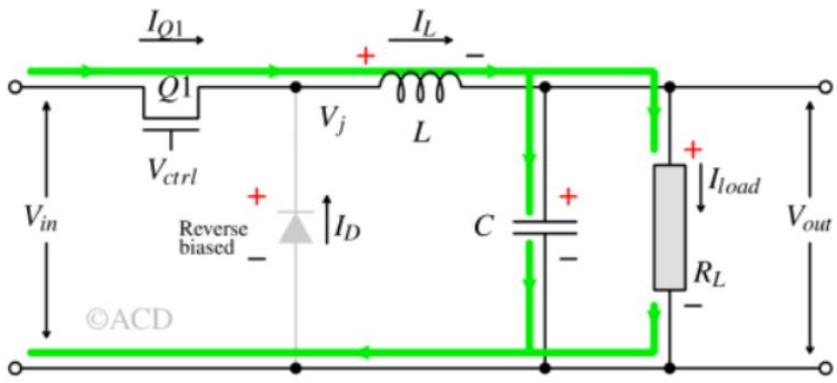
```
import processing.serial.*;
Serial myPort;
String data = "";
String vin = "", vout = "", iout = "", potStatus = "";
// Background image (optional)
PImage bg;
void setup() {
fullScreen();
background(0);
textSize(40);
fill(0, 255, 0);
// Auto-detect ports
println(Serial.list());
myPort = new Serial(this, "COM6", 9600); // <-- replace with your Arduino
port
}
void draw() {
background(0); // default black background
// If you have a background image
// if (bg != null) image(bg, 0, 0, width, height);
// Header
fill(0, 200, 255);
textSize(60);
```

```
textAlign(CENTER);
text("Embedded System Design Laboratory", width/2, 80);
textSize(50);
text("Buck Converter", width/2, 150);
// Box dimensions
float boxW = 600;
float boxH = 120;
float gap = 40;
// Left column
drawBox(100, 250, boxW, boxH, "Input Voltage (Vin): " + vin + " V");
drawBox(100, 250 + boxH + gap, boxW, boxH, "Output Voltage (Vout): " + vout +
" V");
// Right column
drawBox(width - boxW - 100, 250, boxW, boxH, "Output Current (Iout): " + iout
+ " A");
drawBox(width - boxW - 100, 250 + boxH + gap, boxW, boxH, "Potentiometer: " +
potStatus);
// Read serial data
while (myPort.available() > 0) {
data = myPort.readStringUntil(&apos;\n&apos;);
if (data != null) {
data = trim(data);
parseData(data);
}
}
}
// Draw big labeled box
void drawBox(float x, float y, float w, float h, String label) {
stroke(0, 255, 0);
strokeWeight(5);
fill(0, 50); // semi-transparent fill
rect(x, y, w, h, 20); // rounded corners
fill(0, 255, 0);
textSize(40);
textAlign(LEFT, CENTER);
text(label, x + 20, y + h/2);
}
// Parse Arduino serial data
void parseData(String s) {
// Vin: 12.45V Vout: 11.89V Iout: 0.34A Pot: 580 -> Increased
String[] parts = splitTokens(s, " ");
if (parts.length >= 10) {
vin = parts[1].replace("V", "");
vout = parts[3].replace("V", "");
iout = parts[5].replace("A", "");
potStatus = parts[9];
}
}
```

9. Working Principle

When the switch is turned ON, current flows through the inductor, storing energy in its magnetic field and delivering power to the load. When the switch is turned OFF, the inductor releases the stored energy, maintaining current flow to the load through the diode. By varying the duty cycle of the PWM (Pulse Width Modulated) signal that drives the switch, the average output voltage can be controlled. This principle allows the converter to achieve efficient voltage reduction with minimal power loss compared to linear regulators.

Switch in ON

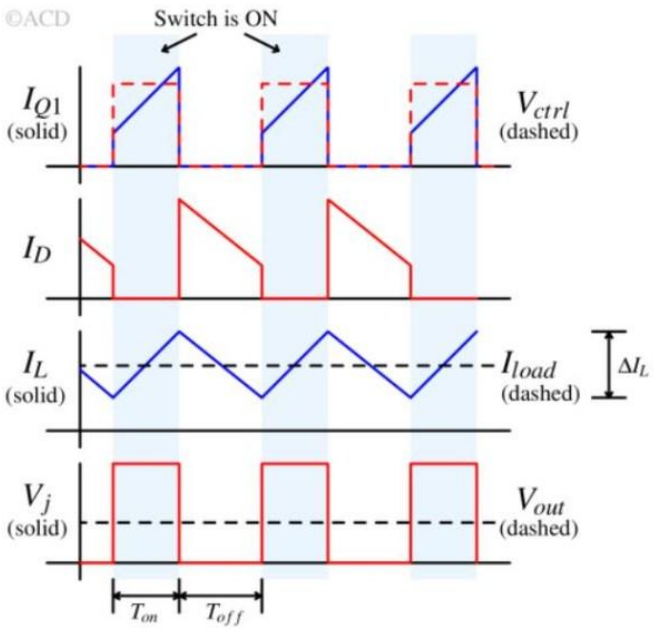


To facilitate monitoring and control, the microcontroller (Arduino or ESP32) plays a pivotal role in acquiring system parameters. The input voltage (V_{in}) and output voltage (V_{out}) are sensed through voltage divider circuits, which scale down the voltage levels to fall within the microcontroller's safe ADC (Analog-to-Digital Converter) input range, typically 0–5V.

This ensures accurate measurement without exceeding the microcontroller's voltage limits. Meanwhile, the output current (I_{out}) is measured using an ACS712 Hall-effect current sensor, which produces a proportional voltage output based on the current flowing through it.

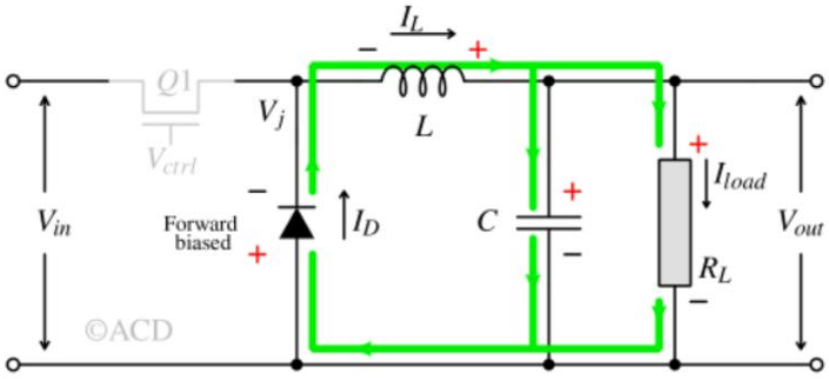
During this phase, current builds up, the inductor stores energy in magnetic field and resists sudden changes in current. Energy from the input source is transferred to the inductor during this ON-period. The voltage across the inductor is the difference between the input and output voltage.

$$\Delta I_L = \frac{V_{in} - V_{out}}{L} \times T_{ON}$$



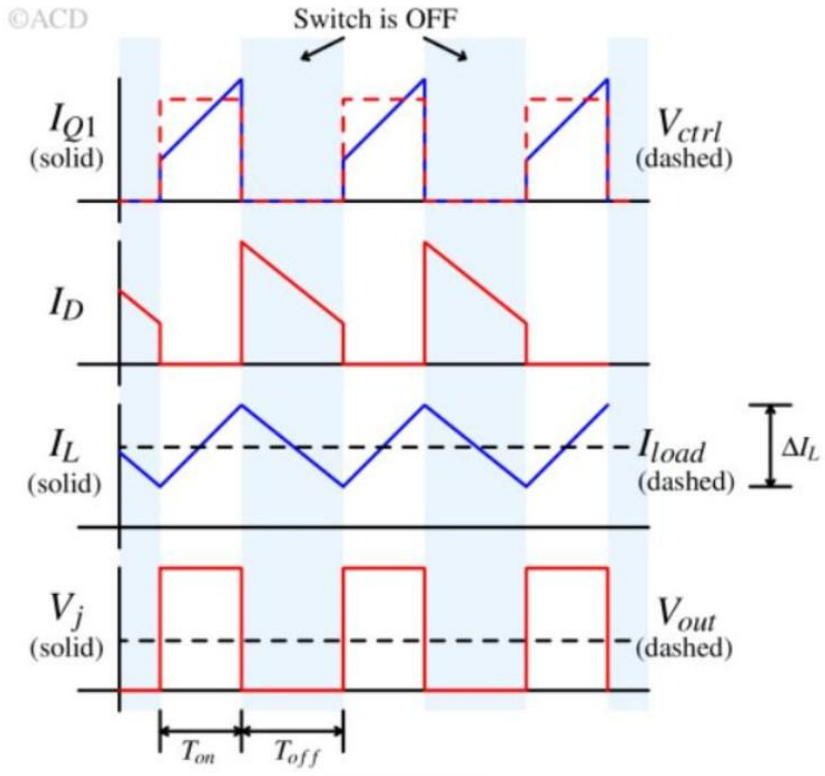
This voltage serves as an input parameter to determine whether the user is increasing or decreasing the desired output voltage. The system then identifies this change as *"Increased"*, *"Decreased"*, or *"Stable"*, providing feedback through both the LCD and the graphical dashboard.

Switch is OFF



After acquiring all sensor data, the microcontroller processes and displays the results locally on a 16x2 I2C LCD. The LCD provides immediate feedback to the user by showing Vin, V_{out}, I_{out}, and the current potentiometer status in real-time. Simultaneously, the microcontroller transmits the same data via serial communication (UART) to a PC-based application developed in processing.

The Processing sketch acts as a graphical dashboard, updating visual elements such as text boxes and numerical displays. This real-time graphical interface gives the user an intuitive and interactive view of the system's operating conditions, providing insights into how changes in the potentiometer or input supply affect the output voltage and current.



$$\Delta \boldsymbol{I_L} = \frac{-\boldsymbol{V_{out}}}{\boldsymbol{L}} \times \boldsymbol{T_{OFF}}$$

The seamless integration between the hardware (buck converter, sensors, and microcontroller) and software (Processing dashboard) establishes a feedback loop where data acquisition, processing, display, and user control occur simultaneously. This coordination ensures smooth operation, high accuracy, and immediate responsiveness to user actions. The design showcases the essence of embedded system engineering—where hardware control meets software visualization—providing a complete learning experience in data acquisition, communication protocols, and real-time monitoring in power electronic systems.

Buck converter equation and duty cycle is derived as follows;

The output voltage is fixed using an IC controller. If the load is fixed, current remains fixed, so as to maintain the same average current to a fixed load. The current through the inductor during the ON period has to be same as the change in OFF period, which implies

$$\Delta I_L(on) = \Delta I_L(off)|$$

$$\because V_{out} < V_{in}$$

$$\frac{V_{in} - V_{out}}{L} T_{on} = -\left(\frac{-V_{out}}{L} T_{off}\right)$$

$$\frac{V_{out}}{V_{in}} = \frac{T_{on}}{T_{on} + T_{off}}$$

Let duty cycle is D, which is :

$$D = \frac{T_{on}}{T_{on} + T_{off}}$$

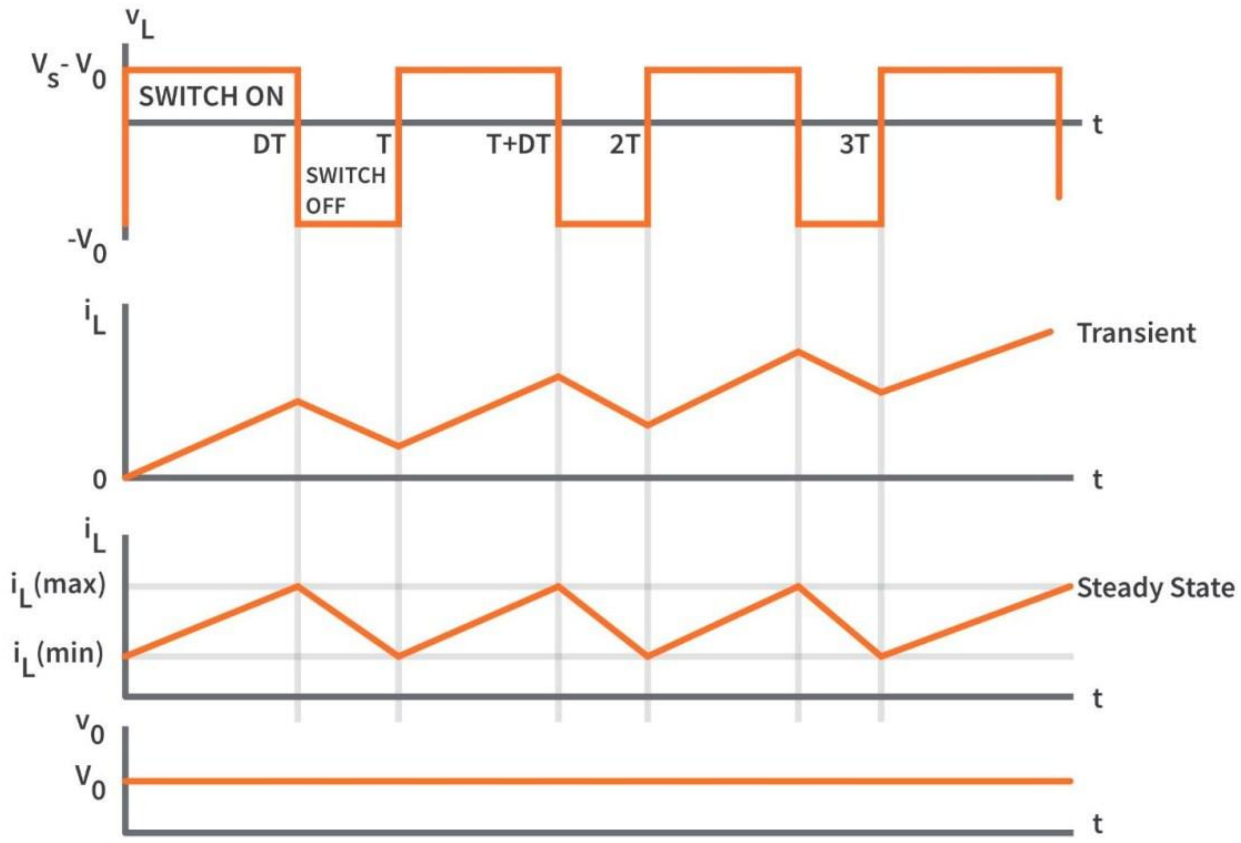
Therefore, the buck equation in terms of duty cycle:

$$\frac{V_{out}}{V_{in}} = D$$

10. Observations

The performance of the buck converter system was analyzed by measuring key electrical parameters under controlled operating conditions. The input voltage was varied manually using a DC power supply, while the output voltage was adjusted through a potentiometer connected to the control section. The corresponding output current was recorded using the ACS712 sensor.

Parameter	Expected Range	Measured Value	Description
V_{in}	30 V (DC)	24 V (DC)	Input voltage to buck converter
V_{out}	0-20 V (DC)	7.97 V (DC)	Output voltage controlled by pot
I_{out}	0-5A	2.32 A	Load current via ACS712



During testing, it was observed that the system responded immediately to potentiometer adjustments, with corresponding changes reflected on both the LCD display and the Processing dashboard. The measurements exhibited high consistency, validating the accuracy of the voltage divider and ACS712 sensor calibration. The graphical dashboard provided an intuitive visualization of system performance, effectively demonstrating the dynamic interaction between user input and converter output.

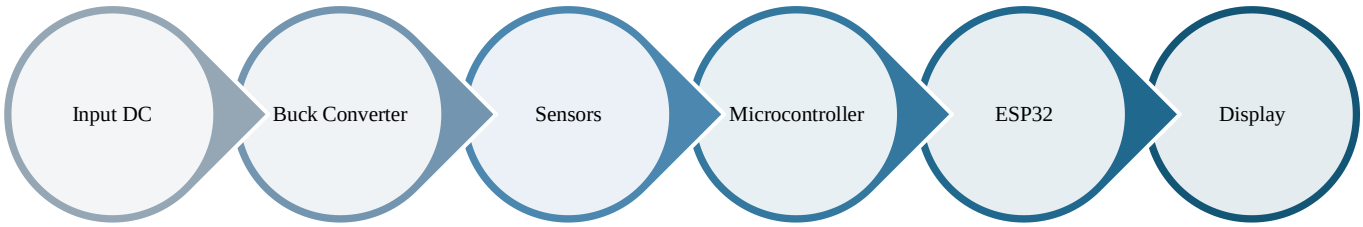
11. Applications

The developed buck converter system serves as a versatile educational and practical platform that bridges the gap between embedded control and power electronics. Its design allows application across various domains where DC voltage regulation and real-time monitoring are essential.

Key Applications:

1. **DC Voltage Regulation:**
   Used for powering microcontrollers, sensors, and low-voltage circuits from a higher DC source such as a solar panel or battery bank.

2. **Embedded System Training Module:**
   Serves as a learning platform for students to understand ADCs, PWM control, data acquisition, and sensor interfacing.
3. **Real-Time Monitoring and IoT Integration:**
   Enables visualization of voltage and current parameters via the dashboard; can be extended with Wi-Fi (ESP32) for IoT-based remote monitoring.
4. **Smart Energy and Automation Systems:**
   Potential for integration into smart grid setups or automated voltage regulators in renewable energy applications.
5. **Laboratory Demonstration Tool:**
   Provides real-time feedback for concepts in energy conversion, control systems, and embedded interfacing.



12. Conclusion

The implemented buck converter with real-time dashboard successfully demonstrates the integration of power electronics, embedded control, sensor interfacing, and graphical visualization into a unified platform.

The system achieves accurate measurement of voltage and current, reliable potentiometer-based control, and seamless data communication between hardware and the Processing-based GUI.

The project not only validates theoretical principles of DC-DC conversion but also strengthens practical understanding of analog signal acquisition, ADC scaling, serial communication, and graphical programming.

The dual-display mechanism—via LCD and Processing—offers both local and remote monitoring capabilities, enhancing usability and system transparency.

In conclusion, this project embodies the principles of embedded system design and power electronics integration. It provides a scalable foundation for further enhancements, such as:

- **IoT-enabled data logging**,
- **Wireless communication using MQTT or Wi-Fi**, and
- **Closed-loop feedback for automated voltage control**.

Through its modular architecture and educational focus, this project stands as a comprehensive demonstration of modern embedded design applied to real-time energy systems, showcasing how hardware-software synergy can bring theoretical concepts to life.

13. References

1. Arduino Official Documentation: https://www.arduino.cc/reference/en/
2. ESP32 ADC Reference: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/adc.html
3. ACS712 Datasheet: Allegro Microsystems, ACS712 Hall Effect Current Sensor
4. Processing Serial Library: https://processing.org/reference/libraries/serial/index.html
5. Voltage Divider Theory: https://www.electronics-tutorials.ws/resistor/voltage-divider.html